```
import akka.actor.typed.Behavior
import akka.actor.typed.scaladsl.Behaviors

object LoggerBehavior {
  def apply(tag: String): Behavior[Unit] = {
    Behaviors.setup { context =>
      context.log.info("spawned LoggerBehavior {}",tag)     ❶
      Behaviors.ignore
    }
  }
}
```

❶ Logs the tag

Finally, you can create a `ShardedDaemonProcess` with an `ActorSystem`, the `LoggerBehavior`, and the `tag`s. Here is a simple approach to create such a `ShardedDaemonProcess` in a cluster. The tags are a fixed variable, whereas normally they would be read from the configuration.

Listing 10.10 Using a `ShardedDaemonProcess`

```
import akka.cluster.sharding.typed.scaladsl.ShardedDaemonProcess

object MiniCluster {

  def main(args: Array[String]): Unit = {
      startup(args(0).toInt)
  }

  def startup(port: Int): Unit = {
    val config = ConfigFactory
      .parseString(s"""
      akka.remote.artery.canonical.port=$port
      """)
      .withFallback(ConfigFactory.load("shardeddeamon"))     ❶

    val system = ActorSystem[Nothing](Behaviors.empty, "MiniCluster", config)

    val tags = Vector("container-tag-1",
                "container-tag-2",
                "container-tag-3")                           ❷

    ShardedDaemonProcess(system)
          .init(
              "loggers",
              tags.size,
              index => LoggerBehavior(tags(index)))          ❸
  }
}
```

❶ Loads the configuration from the shardeddeamon.conf file

❷ Fixed tags, thus parallelism

❸ Initialization of the behavior

Tags are used to determine the number of instances of `LoggerBehavior`s, and they can be used to select a specific tag to read events from the journal. This was exactly what the `SPContainer` example needed to do. With this knowledge about the `ShardedDaemonProcess`, you can understand how to use it with the projection of `SPContainer`.

10.3.5 Back to the SPContainer projection